WhizUnik Command

Trading vs Manufacturer

Two options on the landing page / dashboard — detailed implementation plan with RFB Latex Demo flow (10,000 gloves, demo data)

Item	Detail
Purpose	Add a **Manufacturer** mode alongside existing **Trading** mode without destabilising core WhizUnik. Reuse trading flows; add generic manufacturing modules; configure RFB specifics as data.
Entry point	Public landing (/) shows two cards: **Trading & Distribution** and **Manufacturing**. Post-login dashboard (/app/dashboard) has a persistent **[Trading | Manufacturer]** toggle stored per tenant + user.
Product structure	L1 Core WhizUnik (stable) → L2 Reusable manufacturing modules (generic names) → L3 Customer configuration (RFB lines, stages, shifts, sizes, roles, Tally settings). No RFB code fork.
Stock invariant	PO / PI never create stock — only **confirmed GRN** creates stock IN. SO / invoice never reduce stock — only **confirmed dispatch** creates stock OUT. Draft/cancelled never count.
Demo scope	One sample order of **10,000 gloves** (labelled DEMO DATA). Material shortage → PO → GRN → batch → 2 production updates → factory stoppage → Pending → Hold (blocked) → Release → allocate → dispatch readiness → Tally handoff (simulated, labelled).

How to read this document: §2 is the mode comparison. §3–§4 are the exact flows. §5–§7 are build-ready specs (models, APIs, UI). §8 is the click-by-click demo script. §9–§11 are acceptance, phases and non-goals.

1 Guiding principles (must remain unchanged)

- Existing customer workflows and screens continue to work. Trading mode is byte-for-byte the current product.
- A purchase order or purchase invoice must **not** increase stock. Only a **confirmed GRN** creates stock IN.
- A sales order or sales invoice must **not** reduce stock. Only a **shipped/confirmed dispatch** creates stock OUT.
- Draft and cancelled documents never affect stock: live stock = SUM(confirmed IN) − SUM(confirmed OUT).
- Finance, IRN and e-way bill responsibilities remain as currently implemented unless a later agreed change says otherwise.
- No RFB-specific **if (client === 'RFB')** conditions scattered through the codebase. RFB = configuration + seed data + generic modules.
- Every simulated Tally / compliance step is visibly labelled SIMULATED / SAMPLE EXPORT.

Decision rule for every future RFB request: **Core?** (needed by most customers → build in core) · **Reusable module?** (needed by a class → build generically) · **Setting?** (one tenant → configure as data). If it would change core stock, finance or document behaviour — stop and review impact first.

2 The two options — what each mode contains

2.1 Mode selector behaviour

- **Public landing (/):** two cards — *Trading & Distribution: buy, stock, sell, collect* and *Manufacturing: everything in Trading plus batches, live floor updates, factory issues, quality hold/release*. Cards route to /auth?mode=trading | ?mode=manufacturer and set a default; auth logic itself is unchanged.
- **Post-login (real switch):** persistent toggle in the topbar + dashboard header. Stored as **tenantSettings.businessMode** ('trading' | 'manufacturer' | 'both') with per-user override **user.lastMode**. Sidebar builder **buildNavSections(roles, mode)** filters nav: trading hides Manufacturing; manufacturer shows it. Same URLs, same permissions, no URL fork.
- Demo tenant shows a **DEMO DATA** badge and a **Reset demo** button scoped to that tenant only (clientId = RFB-DEMO).

2.2 Feature matrix

Area	Trading (existing — unchanged)	Manufacturer (reuse + additions)
Dashboard	Command Centre: cash, receivables/payables, sales MTD, procurement, inventory intelligence, funnel	Manufacturer Command: open orders, batch progress, good/reject, factory issues, held qty, released-ready qty. Every tile links to its source list
Catalogue	app/products: SKU, pricing, GST, reorder, margin	Reuse as-is. Configure glove sizes as *size* values, category = Latex Gloves. No schema change
Customers / Suppliers	app/debtors, app/suppliers	Reuse as-is. Seed 1 demo customer + 1 latex supplier
Purchase flow	procurement-workbench: proforma → PO → PI → GRN → stock IN	Reuse 100% for raw-material shortage (latex, chemicals). Stock rule untouched
Sales flow	sales-workbench: quotation → SO → dispatch → invoice + NOA	Reuse 100%. SO for 10,000 gloves. Dispatch confirm adds one generic guard: batch must be Released
Inventory	warehouse-workbench, stock-movement ledger, forecast	Reuse ledger. Finished gloves enter via existing IN path (reason = 'Production receipt', linked to batch). No parallel ledger
Production Batch (NEW)	Hidden	ManufacturingBatch → SO + product + target qty + line. planned → in_progress → completed → closed
Live updates (NEW)	Hidden	ProductionUpdate: incremental entries (stage, qty since last, rejects, line status, remarks, user, time). Idempotency key blocks double-count
Factory issues (NEW)	Hidden	FactoryIssue: stoppage / maintenance / staffing / utility / material. Line + affected batch + owner + expected resolution + open/closed
Quality (NEW)	Hidden	qualityStatus Pending → Hold → Released, separate from production status. Only quality role can Release. Pending/Hold cannot be allocated
Dispatch readiness	Soft stock check (warn)	Hardened: only Released stock allocatable. Pending/Hold → blocked message + link to quality page
Finance / IRN / e-way	finance-workbench, journals, e-way bill	Reuse unchanged
Tally	tallyVoucherRef display	Same + SIMULATED sample export button on manufacturer dashboard. No new live integration

3 Trading flow (reference — do not rebuild)

Supplier proforma → Purchase Order → Purchase Invoice → **GRN (confirm = stock IN)** → stock. Quotation → Sales Order → **Dispatch (confirm = stock OUT)** → Sales Invoice → NOA / reminders / payment. Manual movements (opening, adjustment, damage, samples, customer/supplier returns) start as draft and affect stock only on confirm. System movements from GRN/dispatch are immutable from the ledger (manage from the source document). Cancel reverses its own effect (GRN cancel → reversing OUT + PO revoke; dispatch cancel → reversing IN + SO revoke). Maker–checker gates (quote→SO needs approval; proforma→PO needs approval; invoice issue needs confirmed SO; PI needs approved+sent PO; over-receipt needs admin/checker flag) stay enforced server-side.

Document	Stock effect	When
Purchase Order (PO)	None	—
Supplier proforma (purchase)	None	—
Purchase Invoice (PI)	None	—
GRN	Stock IN (accepted qty)	On Confirm only
Quotation / Sales Order / Sales Invoice	None	—
Dispatch note	Stock OUT (dispatched qty)	On Confirm only
Customer return	Stock IN	On Record Return
Manual movement	IN / OUT	On Confirm only

4 Manufacturer flow (reused functions + additions)

4.1 End-to-end chain

Customer order (SO, 10,000 gloves, due date) → material check → **shortage** → **PO** → **GRN** → **raw-material stock IN** (existing) → **create ManufacturingBatch** (SO + product + target + line) → **live ProductionUpdates** (incremental, idempotent; dashboard progress/good/rejects) → **FactoryIssue** if stoppage (owner, line, batch, open/closed) → **complete batch** → **quality Pending** → **Hold** → **allocation attempt BLOCKED** → **quality Release (authorised role)** → **allocate Released stock** → **dispatch readiness** → dispatch confirm (existing stock OUT) → invoice (existing) → **Tally handoff (simulated, labelled)**.

4.2 What is reused vs what is new

Step	Reuse (exact route / model)	New (generic module)
Order in	goods-sales-orders (SO-XXXXXXXX), debtors	—
Material in	goods-purchase-orders, goods-receipts confirm, stock-movements IN	—
Make	—	manufacturing-batches + production-updates (recalculate batch & order progress)
Interruptions	—	factory-issues (owner, expected resolution, open/closed)
Quality gate	—	qualityStatus Pending/Hold/Released + reviewer/time/remarks; allocation guard
Finished stock	stock-movements (reason = Production receipt, linked batchId)	Derived views: batch qty / rejected / released / available
Ship & bill	goods-dispatches confirm (stock OUT), invoices, e-way, journals	Dispatch-readiness checklist (balance + document checklist)
Report	Existing reporting + forecast (flows through naturally)	manufacturer-overview aggregation; tally-export-preview (simulated)

4.3 Key invariants for manufacturing

- Quality status is **separate** from production status. A batch can be production-completed yet quality-Held — and still unshippable.
- Good output + rejects = recorded production movement per update (validated server-side; rejects never silently dropped).
- Held / Pending batches **cannot** be allocated or dispatched — enforced in **both** allocate API and dispatch confirm (assertBatchReleasable).
- Only an authorised quality role (quality_manager / checker / admin) can Release; every transition writes reviewer, timestamp, remarks + audit log.
- Dashboard counts come from saved records (manufacturer-overview) and each tile deep-links to its source list.
- Refreshing the page never loses an update (server-persisted immediately); re-submitting the same idempotency key never double-counts.

5 Data design (generic names — never Rfb*)

5.1 manufacturing-batch

id, clientId, batchNumber (BATCH-XXXXXXXX), salesOrderId, salesOrderNumber, productId, sku, productName, targetQty, lineId, shiftId, status (planned | in_progress | completed | closed), qualityStatus (Pending | Hold | Released, default Pending), qualityReview[] {by, at, remarks}, producedGoodQty (derived), rejectedQty (derived), releasedQty, allocatedQty, availableQty (derived = released − allocated), notes, createdBy, timestamps, isDemoData.

5.2 production-update (incremental, idempotent)

id, clientId, batchId, stage (config-driven: e.g. Dipping / Washing / Drying / Stripping / Packing), qtySinceLastUpdate (> 0), rejectedQty (≥ 0), lineStatus (running | stopped | maintenance), remarks, createdBy, createdAt, **idempotencyKey** (UUID minted when the form opens; unique index per client — duplicate POST returns the original row, credits nothing). Server re-sums parent batch after each insert; concurrent confirms use conditional writes.

5.3 factory-issue

id, clientId, type (stoppage | maintenance | staffing | utility | material), lineId, affectedBatchId, title, description, owner (userId), expectedResolutionAt, status (open | closed), closedAt, closedBy, createdBy, timestamps. Dashboard shows owner + affected line/batch + status.

5.4 manufacturing-config (per tenant — this is where RFB lives)

clientId, lines[] {id, name} (e.g. Line-1, Line-2), shifts[] {id, name} (Day / Night), stages[] (dipping → packing), sizes[] (S / M / L / XL as product size values), qualityRoles[] (who may Release), tallyMode (simulated | live-test), dashboardLabels. Seed script writes the RFB values here — zero code branches.

5.5 Finished-stock view (no parallel ledger)

Batch quantity / rejected / released / available are **derived**: production updates feed producedGoodQty + rejectedQty; Release sets releasedQty; allocate sets allocatedQty; production receipts and dispatch issues flow through the existing **stock-movement** table (linked via batchId + linkedDocumentType). Live sellable stock for a SKU remains SUM(confirmed IN) − SUM(confirmed OUT).

6 API, routes and navigation

6.1 New APIs (same auth + tenant middleware as existing)

Method + path	Purpose	Guards
GET/POST /manufacturing-batches	Create batch (SO + product + target + line); list	SO must be confirmed; product active
GET/PUT /manufacturing-batches/:id	Batch detail / edit while planned|in_progress	Lines with released qty are protected
POST /manufacturing-batches/:id/complete	production status → completed (quality stays Pending)	All updates persisted; idempotent
POST /manufacturing-batches/:id/hold|release	qualityStatus transitions + reviewer/time/remarks	Release: quality role only; audit logged
GET/POST /production-updates	Incremental update (idempotencyKey required)	qty>0; duplicate key returns original
GET/POST /factory-issues; POST /:id/close	Raise / resolve stoppages	Close records closedBy/At
POST /manufacturing-batches/:id/allocate	Allocate Released stock to SO	409 BLOCKED unless Released; qty ≤ available
GET /manufacturer-overview	Dashboard aggregation (orders, progress, issues, held, ready)	Server-computed from saved records
GET/PUT /manufacturing-config	Lines, shifts, stages, roles, Tally mode	Admin / ops
GET /tally-export-preview	Simulated CSV/JSON handoff	Always labelled SIMULATED unless live-test verified

6.2 Frontend routes (same workbench pattern as Procurement / Sales / Warehouse)

- New shell **app.manufacturer-workbench.tsx** with tabs: *Overview* | *Batches* | *Production Updates* | *Factory Issues* | *Quality* | *Dispatch Readiness* | *Tally Export*. Batches/Updates/Issues/Quality are new pages; Dispatch Readiness embeds the existing dispatch flow plus the release guard and checklist.
- Mode filter added to **app.tsx buildNavSections(roles, mode)**: trading hides the Manufacturing entry; manufacturer shows it. Route wall extended with manufacturing paths per role (quality pages require quality role).
- Public landing **index.tsx**: two cards (Trading / Manufacturing) setting ?mode for the post-login default. Dashboard **app.dashboard.tsx** branches: existing Command Centre vs new Manufacturer Command (same URL).

7 Dashboard specifications

7.1 Trading dashboard — unchanged

Current Command Centre (cash, receivables/payables with ageing, sales MTD, purchases MTD, gross profit, inventory value/SKUs/low/out, business-performance chart, sales funnel Enquiry → Paid, procurement delayed POs, operations dispatch queue, cash-flow command, insights, attention list, cross-functional priorities). No edits.

7.2 Manufacturer dashboard — new (all tiles link to source)

Tile	Shows	Source / link
Order card	SO number, 10,000 gloves, due date, balance = ordered – allocatedReleased	goods-sales-orders detail
Batch progress	producedGood / target bar, good output, rejects, % complete	manufacturing-batch detail

Tile	Shows	Source / link
Factory issues	Open count; each: line, affected batch, owner, expected resolution, open/closed	factory-issues list
Quality gate	Pending (amber) / Hold (red, blocked) / Released (green, ready qty); Allocate button demonstrates BLOCKED on Hold	quality tab + audit
Dispatch readiness	Checklist: Released ≥ balance? SO confirmed? Invoice? E-way? Tally export?	dispatches + invoices
Tally handoff	SIMULATED badge, sample export download, status log	tally-export-preview

8 RFB demo script — click by click (10,000 gloves, DEMO DATA)

#	Action	Expected result
1	Log in → switch to Manufacturer → confirm DEMO DATA badge	Seed tenant RFB-DEMO isolated; Reset demo visible
2	Open the sample customer order	SO shows 10,000 gloves + due date + balance 10,000
3	Show material shortage → open/create PO → record receipt (GRN confirm)	Raw-material stock IN via existing GRN rule; PO status partially/fully_received derived
4	Create production Batch-B1 (SO + glove product + target 10,000 + Line-1)	Batch planned → in_progress; appears on dashboard at 0%
5	Enter production update #1 (e.g. 4,000 good + 200 reject, Dipping)	Dashboard ~40%, good 4,000, rejects 200
6	Enter production update #2 (e.g. 5,000 good + 150 reject)	Dashboard ~90% with no double-count; refresh + resubmit-safe
7	Record factory stoppage (Line-1, Batch-B1, owner, expected resolution)	Open-issue tile shows owner + line + batch + status open
8	Complete batch (quality Pending) → set Hold → click Allocate	Allocation BLOCKED with message + audit entry; dispatch confirm also refuses
9	As authorised quality user: Release batch → allocate to order	Released qty available; balance falls; dispatch readiness turns green
10	Confirm dispatch (existing flow) → show invoice + Tally handoff	Stock OUT on dispatch confirm only; Tally card labelled SIMULATED sample export

9 Acceptance checks (must pass before showing RFB)

- Refreshing the page loses no production or factory update (server-persisted).
- Submitting an update twice does not double-count (idempotency key).
- Good output + rejects equals the recorded production movement per update.
- Held or Pending batches cannot be allocated or dispatched (both APIs refuse with 409).
- Only an authorised quality role can Release (others get 403 + audit).
- Every factory issue shows owner, affected line/batch and open/closed status.
- Dashboard counts match underlying records and open the correct source lists.
- Existing GRN / dispatch / stock balances still follow current rules (regression on trading flows).
- Demo tenant resets without touching other tenants.

- Every simulated Tally / compliance step is visibly labelled.

10 Build phases

Step	Developer action	Output
1. Inspect	Map existing routes, APIs, models, permissions, stock calculations against the workflow guide	Reuse / Configure / Build / Defer matrix (this document)
2. Isolate	Tenant feature flag businessMode + manufacturing-config + seed/reset script (seed-rfb-demo.mjs)	RFB demo cannot alter other customer data
3. Reuse	Use current product / supplier / order / GRN / inventory / dispatch flows; add mode toggle + workbench shell	No duplicate core workflows
4. Extend	Add generic ManufacturingBatch, ProductionUpdate, FactoryIssue (+ quality transitions) only where missing	Reusable module capability
5. Configure	RFB products, stages, lines, shifts, sizes, roles, dashboard labels, Tally simulated mode as data	RFB behaviour without code fork
6. Verify	Run full 10k-glove flow + regression on core trading paths + acceptance checks	Working demo with evidence core is unchanged

11 Keep out of the demo build

Do not add detailed formulations/BOM explosion, automatic MRP, machine/IoT integration, advanced laboratory test plans, sterilisation management, full export compliance, advanced costing, or a new live Tally integration before RFB agrees the pilot scope. Record these as later discovery items.

Roles & gates summary

Role	Can
Client (maker)	Create/edit drafts, submit, mark PO sent, draft receipts/dispatches, record production updates & issues
Quality (checker / quality_manager)	Hold / Release batches, approve quotations & proformas & POs, allow over-receipt with flag
Treasury / Admin	Record payments, fund proformas, full access, reset demo tenant
Sales rep	Read-only on PO/SO/dispatch; write quotations; view manufacturer overview

Owner: Product and Engineering. Scope: RFB demo preparation. Status: proposed implementation guide. Next: approve → Phase 1 (mode toggle + config + seed) → Phase 3 (batches / updates / issues / quality).